

AULA 4

Laboratório: CRUD Completo

Disciplina: CK0207 — Desenvolvimento de Software para Web

Professor: Dr. Fernando Antônio Mota Trinta

Universidade Federal do Ceará (UFC)

Projeto do curso: `nest-api-curso` (continuação da Aula 3)

Autor: Samuel Sales Furtado

Objetivos de Aprendizagem

- Completar o CRUD com as operações de Update e Delete
- Tratar erros com exceções HTTP apropriadas
- Reaproveitar um DTO de criação num DTO de atualização parcial
- Consolidar boas práticas de API REST vistas desde a Aula 2

Duas pontas ficaram soltas desde a Aula 2: buscar um usuário inexistente devolve vazio, em vez de um erro de verdade; e nunca implementamos atualizar ou remover um usuário. Esta aula fecha essas duas pontas — o CRUD **completo**: Create, Read, Update, Delete, com tratamento de erro apropriado em cada operação.

1. Operações CRUD Completas

As rotas que faltavam: `PATCH /users/:id`, para atualizar parcialmente, e `DELETE /users/:id`, para remover.

O verbo é `PATCH`, não `PUT` — `PUT` normalmente espera o recurso inteiro substituído, enquanto `PATCH` espera apenas os campos que o cliente deseja alterar.

1.1 PartialType

```
export class UpdateUserDto extends PartialType(CreateUserDto) {}
```

`PartialType` pega todas as propriedades do `CreateUserDto` — com os decorators de validação já definidos na Aula 3 — e torna cada uma opcional automaticamente. Sem isso, seria necessário reescrever `@IsString()`, `@IsEmail()` e acrescentar `@IsOptional()` em cada campo manualmente, trabalho repetido e propenso a inconsistência. `PartialType` vem do pacote `@nestjs/mapped-types` (ou de `@nestjs/swagger`, equivalente, se o projeto usar Swagger — Aula 11).

1.2 DELETE

O `DELETE` não recebe corpo de requisição, apenas o `id` na URL, e por convenção devolve `204 No Content` — sucesso, sem corpo de resposta. É necessário usar `@HttpCode(204)` explicitamente, já que o padrão do Nest para qualquer método que não seja `@Post()` é `200`.

2. Tratamento de Erros

```
findOne(id: number): User {
  const user = this.users.find((u) => u.id === id);
  if (!user) {
    throw new NotFoundException(`Usuário ${id} não encontrado`);
  }
  return user;
}
```

`NotFoundException` é uma das várias exceções prontas do Nest. Todas — `NotFoundException`, `BadRequestException`, `ConflictException`, entre outras — herdam de `HttpException`, e o Nest já sabe transformar qualquer uma delas em status code + corpo JSON, sem configuração adicional.

Importante: essa exceção deve ser lançada dentro do **Service**, não apenas no Controller — e o Nest propaga corretamente o erro até virar resposta HTTP. Isso é coerente com o princípio definido desde a Aula 1: quem decide se um dado existe é a camada de regra de negócio, não o controller.

Regra geral: em qualquer lugar do código onde a resposta seria `null`, `undefined`, ou um array vazio representando "não encontrei", a resposta correta normalmente é lançar uma exceção HTTP apropriada. Isso vale igualmente para `update` e `remove`.

3. Boas Práticas para APIs REST

Verbo	Ação	Status de sucesso
GET	Ler (nunca altera estado)	200
POST	Criar	201
PATCH	Atualizar parcialmente	200
DELETE	Remover	204 (sem corpo)

Erros comuns: 404 quando o recurso não existe, 400 quando o dado enviado é inválido.

Use sempre exceções específicas do Nest em vez de try/catch genérico espalhado pelo código, ou checagens manuais que escondem o problema em vez de sinalizá-lo. E a regra que abre este curso continua valendo: **o controller nunca acessa o "banco" diretamente** — mesmo com CRUD completo e tratamento de erro, essa separação de responsabilidades permanece.

Laboratório — Passo a Passo ## Passo 1 — Criar o `UpdateUserDto`

```
npm install @nestjs/mapped-types
```

```
// src/users/dto/update-user.dto.ts
import { PartialType } from '@nestjs/mapped-types';
import { CreateUserDto } from '../create-user.dto';

export class UpdateUserDto extends PartialType(CreateUserDto) {}
```

Passo 2 — Adicionar `update` e `remove` no Service

```
findOne(id: number): User {
  const user = this.users.find((u) => u.id === id);
  if (!user) throw new NotFoundException(`Usuário ${id} não encontrado`);
  return user;
}

update(id: number, data: UpdateUserDto): User {
  const user = this.findOne(id);
  Object.assign(user, data);
  return user;
}

remove(id: number): void {
  const index = this.users.findIndex((u) => u.id === id);
  if (index === -1) throw new NotFoundException(`Usuário ${id} não encontrado`);
  this.users.splice(index, 1);
}
```

Passo 3 — Adicionar `PATCH` e `DELETE` no Controller

```
@Patch('/:id')
update(@Param('id') id: string, @Body() updateUserDto: UpdateUserDto) {
  return this.userService.update(+id, updateUserDto);
}

@Delete('/:id')
@HttpCode(HttpStatus.NO_CONTENT)
remove(@Param('id') id: string) {
  this.userService.remove(+id);
}
```

Passo 4 — Testar tudo

```
curl -X PATCH http://localhost:3000/users/1 \  
-H "Content-Type: application/json" \  
-d '{"name":"Ana Paula Souza"}'
```

Esperado: só o `name` muda, `email` permanece igual.

```
curl -i http://localhost:3000/users/99
```

Esperado: status `404` .

```
curl -i -X DELETE http://localhost:3000/users/1
```

Esperado: status `204` , sem corpo.

Verificação de Conformidade

- ✔ `NotFoundException` , `BadRequestException` , `ConflictException` herdando de `HttpException` — conferido em docs.nestjs.com/exception-filters .
- ✔ Exceções lançadas dentro de `providers/services` propagadas corretamente — conferido em exemplos oficiais de `Exception Filters`.
- ✔ `PartialType` disponível em `@nestjs/mapped-types` — conferido na documentação oficial sobre reaproveitamento de DTOs em `CRUD`.
- ✔ `@HttpCode(HttpStatus.NO_CONTENT)` sobrescrevendo o status padrão — conferido em docs.nestjs.com/controllers .